
shieldGPT

Project Plan and System Architecture

A browser extension for preventing unintended secret exposure when interacting with ChatGPT

Project Type	Browser extension / client-side security tool
Primary Problem	Users may accidentally paste API keys, private keys, tokens, credentials, or other secrets into an AI chat.
Proposed Solution	Detect suspicious secret patterns locally before a message is submitted and ask the user for confirmation.
Technology	Chrome Extension Manifest V3, JavaScript, regular expressions, JSON configuration
Target Platform	ChatGPT web application (chatgpt.com and chat.openai.com)

Prepared from the supplied shieldGPT implementation.

1. Project Overview

shieldGPT is a lightweight browser extension designed to reduce the risk of users unintentionally exposing sensitive information while using ChatGPT. The extension runs in the browser and inspects the text entered into the ChatGPT composer immediately before submission.

The core idea is simple: before a message is sent, shieldGPT compares the message against a configurable collection of regular-expression patterns. If one or more patterns match, the extension identifies the type of possible exposure and presents a confirmation dialog. The user can then cancel the submission or explicitly choose to continue.

The supplied implementation is client-side. It loads detection rules from **regexes.json**, reads the current composer content, performs local pattern matching, and intercepts both keyboard-based and button-based submission.

Problem Statement

Modern software development frequently involves copying configuration snippets, logs, environment variables, credentials, source code, and deployment information into AI assistants. A user may unintentionally include a secret such as an API key or private key. shieldGPT addresses the prevention point immediately before submission, giving the user an opportunity to recognize and stop the exposure.

Project Objectives

- Detect common secret and credential formats in text entered into ChatGPT.
- Warn the user before a suspicious message is submitted.
- Support both pressing Enter and clicking the Send/Submit button.
- Keep detection local to the browser for the core workflow.
- Use a separate JSON file so detection patterns can be extended without rewriting the main content script.
- Provide a small, understandable implementation that can be developed and tested incrementally.

2. Brief Problem Research and Scope

The project addresses a practical software-security problem: accidental disclosure of credentials and other secrets through copy/paste and developer tooling. Common secret formats are often recognizable by structure—for example, private-key headers, provider-specific token prefixes, webhook URLs, or API-key formats. This makes regular expressions a practical first-line detection mechanism for a small browser extension.

The project does **not** attempt to prove that a detected value is valid, active, or currently exploitable. A regular-expression match is treated as a warning signal rather than definitive proof of a secret. False positives are therefore possible.

The scope is intentionally limited to preventing accidental submission in the ChatGPT web interface. It does not inspect network traffic, communicate with external security services, rotate credentials, revoke exposed keys, or guarantee detection of every possible secret.

In Scope

- Chrome Manifest V3 extension structure.
- Injection of a content script into ChatGPT pages.
- Loading detection rules from **regexes.json**.
- Reading text from textarea or contenteditable composer elements.
- Matching text against configured regular expressions.

- Displaying categories of detected patterns.
- Blocking submission when the user rejects the warning.
- Allowing submission when the user explicitly confirms.
- Supporting Enter-based and Send/Submit-button submission paths.

Out of Scope

- Server-side storage or processing of user messages.
- Automatic secret revocation or rotation.
- Guaranteeing zero false positives or zero false negatives.
- Inspecting arbitrary websites outside the configured ChatGPT hosts.
- Replacing a full DLP system.
- Validating whether a detected credential is active.

3. Requirements Analysis

Functional Requirements

ID	Requirement	Description
FR-01	Load detection rules	Load regex definitions and titles from regexes.json using the Chrome extension resource URL.
FR-02	Initialize detection	Compile each configured pattern into a JavaScript RegExp object.
FR-03	Read composer content	Obtain text from a textarea or contenteditable ChatGPT composer.
FR-04	Detect possible exposure	Test the composer text against all loaded patterns.
FR-05	Identify categories	Collect matched pattern titles and remove duplicate titles.
FR-06	Warn the user	Display a warning containing the detected exposure categories before submission.
FR-07	Block on rejection	Prevent submission if the user rejects the warning.
FR-08	Allow on confirmation	Allow submission if the user confirms.
FR-09	Handle Enter	Intercept Enter in a composer while allowing Shift+Enter to remain a newline action.
FR-10	Handle Send button	Inspect clicks on buttons whose label, test identifier, or text indicates Send or Submit.
FR-11	Handle empty messages	Allow empty/whitespace-only composer content without scanning.
FR-12	Report loading failures	Log an error if regexes.json cannot be loaded or parsed.

Non-Functional Requirements

ID	Quality Attribute	Description
NFR-01	Performance	Scanning should be fast enough to occur immediately before submission without noticeable delay for normal message sizes.
NFR-02	Privacy	The core detection process should operate locally and should not transmit composer text to a project backend.
NFR-03	Usability	Warnings should clearly state what categories were detected and give a simple continue/cancel decision.
NFR-04	Maintainability	Detection patterns should be stored separately from the main JavaScript logic.

ID	Quality Attribute	Description
NFR-05	Compatibility	The extension should follow Chrome Manifest V3 requirements and support the targeted ChatGPT hosts.
NFR-06	Reliability	Failure to load the detection list should not crash the page; the extension should log the problem.
NFR-07	Security	The extension should minimize permissions and avoid sending detected values or message contents to third parties.
NFR-08	Extensibility	New detection patterns should be addable through regexes.json.
NFR-09	Transparency	The warning should describe matches as possible exposure indicators, not guaranteed secrets.

4. System Architecture

The architecture is intentionally small and modular. The browser hosts the extension, the content script runs in the ChatGPT page, and the JSON file supplies the detection rules. There is no backend in the current design.

High-Level Architecture

Chrome Extension			
manifest.json	content.js	regexes.json	logo.png
Extension configuration	Detection + submission interception	Detection rules	Extension icon
	↓	↓	
	ChatGPT Web Page		
	Composer → scan → warning → allow/block → ChatGPT submission		

Figure 1. High-level shieldGPT architecture and principal interaction path.

Component Responsibilities

Component	Responsibility
manifest.json	Defines Manifest V3 metadata, ChatGPT host permissions, content-script injection, accessible regex resources, and icons.
content.js	Contains application logic: loading patterns, reading composer text, matching patterns, warning the user, and intercepting submission.
regexes.json	Stores detection titles and regular expressions for provider-specific tokens, private keys, webhooks, generic API keys/secrets, and other patterns.
ChatGPT page	Provides the composer and submission controls that the content script monitors.
User	Makes the final decision when a possible secret exposure is detected.

Data Flow

- 1. Chrome loads the extension according to manifest.json.
- 2. content.js starts at document_start on the configured ChatGPT hosts.
- 3. content.js requests regexes.json using chrome.runtime.getURL().
- 4. Each JSON Regex string is compiled into a JavaScript RegExp object.

- 5. The user enters a message in the ChatGPT composer.
- 6. The user presses Enter or clicks a Send/Submit button.
- 7. content.js extracts the current composer text.
- 8. The text is checked against every loaded detection pattern.
- 9. If no pattern matches, submission is allowed.
- 10. If a pattern matches, shieldGPT shows the detected categories and asks for confirmation.
- 11. Rejecting the warning prevents the event from reaching ChatGPT; confirming allows normal submission.

5. Core Design Strategy

The implementation follows a **pre-submission interception** strategy. Instead of modifying ChatGPT's internal application logic, shieldGPT observes user interactions at the browser document level. The event listener runs during the capture phase, giving the extension an opportunity to stop the event before normal page handlers process it.

Detection Pipeline

Input	Processing	Decision	Result
Composer text	Regex scan	No match	Allow submission
Composer text	Regex scan	Match found	Warn user → allow or block

Important implementation decisions

- **Separate rule data:** regexes.json keeps detection definitions independent from application logic.
- **Duplicate removal:** matched titles are converted to a Set and back to an array so the same category is not displayed repeatedly.
- **Shift+Enter handling:** Shift+Enter is deliberately ignored so users can create a newline without triggering pre-send confirmation.
- **Two submission paths:** both keyboard and button interactions are monitored.
- **Capture phase:** listeners use capture=true so shieldGPT can intervene early.
- **Local processing:** message text is processed in the page rather than sent to a shieldGPT server.

6. Implementation Overview

The supplied source consists of content.js, manifest.json, and regexes.json. The excerpts below demonstrate the main design elements.

A. Loading Detection Rules

```
async function loadRegexes() {
  const response = await fetch(
    chrome.runtime.getURL("regexes.json")
  );

  const data = await response.json();

  regexes = data.map((item) => ({
    regex: new RegExp(item.Regex, "i"),
    title: item.Title
  }));

  loaded = true;
}
```

This retrieves the extension rule file, parses the JSON array, and converts each string pattern into a JavaScript regular expression. The loaded flag prevents scanning before initialization has completed.

B. Reading the Composer

```
function getComposerText() {
  const textarea = document.querySelector("textarea");

  if (textarea && textarea.value.trim()) {
    return textarea.value;
  }

  const editor = document.querySelector(
    '[contenteditable="true"]'
  );

  if (editor) {
    return editor.innerText || editor.textContent || "";
  }

  return "";
}
```

The function supports both textarea and contenteditable editing mechanisms.

C. Detection

```
function findExposures(text) {
  if (!loaded || !text) {
    return [];
  }

  const found = [];

  for (const item of regexes) {
    item.regex.lastIndex = 0;

    if (item.regex.test(text)) {
      found.push(item.title);
    }
  }

  return [...new Set(found)];
}
```

Each configured pattern is tested against the message. Only the category title is returned to the warning, rather than displaying the matched secret itself.

D. Blocking a Submission

```
const allowed = checkBeforeSend();

if (!allowed) {
  event.preventDefault();
  event.stopImmediatePropagation();

  console.log(
    "shieldGPT: message blocked by user."
  );
}
```

When the user rejects the confirmation, `preventDefault()` cancels the default action and `stopImmediatePropagation()` prevents later handlers from processing that event.

7. Detection Rule Strategy

The supplied `regexes.json` contains patterns representing multiple categories of sensitive information. These are best described as **possible exposure indicators**, because pattern matching alone cannot establish that a value is authentic or active.

Category	Examples from supplied rules
Cloud and infrastructure	AWS API keys, AWS AppSync keys, Amazon MWS tokens, Google Cloud/service-account indicators.

Category	Examples from supplied rules
Source-control credentials	GitHub-related tokens and authentication credentials.
Communication platforms	Slack tokens and webhook URLs; Telegram bot API keys; Twitter-related patterns.
Payment and commerce	Stripe keys, Square tokens, PayPal/Braintree access tokens, Picatic API keys.
Developer/service credentials	MailChimp, Mailgun, Heroku, Twilio, and generic API-key/secret patterns.
Cryptographic material	RSA, DSA, EC, and PGP private-key block headers.
Authentication artifacts	JSON Web Token indicators and password-in-URL patterns.

The rule set is extensible. Future versions can add patterns, improve patterns, introduce severity levels, or attach remediation guidance.

8. Development Roadmap and Timeline

The assignment specifies approximately 30–35 hours. The following roadmap allocates 31 hours across planning, implementation, testing, documentation, and final review.

Phase	Activity	Effort	Deliverable
Phase 1	Problem research & scope	3 h	Problem definition, scope, target users, assumptions
Phase 2	Requirements analysis	4 h	Functional/non-functional requirements and acceptance criteria
Phase 3	Architecture & design	4 h	Component design, data flow, interaction model
Phase 4	Extension foundation	4 h	Manifest V3, host configuration, resource loading
Phase 5	Detection engine	5 h	Regex loading, composer extraction, matching and categorization
Phase 6	Submission protection	4 h	Enter/click interception, warning dialog, block/allow logic
Phase 7	Testing & refinement	4 h	Positive/negative tests, false positives, browser behavior
Phase 8	Documentation & final review	3 h	Architecture documentation, diagram, cleanup

Total planned effort: 31 hours.

Milestones

- M1 — Scope and requirements approved.
- M2 — Architecture and data-flow design completed.
- M3 — Extension loads successfully on target ChatGPT pages.
- M4 — Secret-pattern detection works against controlled test cases.
- M5 — Submission can be blocked or allowed based on the user's decision.
- M6 — Regression testing and documentation completed.
- M7 — Final PDF and project package ready for human review.

9. Testing Strategy

Testing should verify both correct detection and correct non-detection. Because excessive warnings would make the extension unusable, false-positive testing is as important as positive detection testing.

ID	Scenario	Test	Expected Result
T-01	Known matching token	Paste a controlled value matching one configured pattern.	Correct category is reported.
T-02	Private-key header	Paste a controlled BEGIN RSA/DSA/EC/PGP header.	Cryptographic-material warning appears.
T-03	Normal text	Enter an ordinary question with no secret-like value.	Message is sent without a warning.
T-04	Multiple matches	Use controlled strings matching two or more categories.	All distinct matching categories are shown.
T-05	Cancel	Trigger detection and reject confirmation.	Submission is blocked.
T-06	Confirm	Trigger detection and accept confirmation.	Normal submission proceeds.
T-07	Shift+Enter	Press Shift+Enter in the composer.	Newline behavior remains unaffected.
T-08	Button submission	Click Send after entering a controlled matching value.	Detection runs before submission.
T-09	Empty input	Attempt submission with no meaningful text.	Scanner does not create a false warning.
T-10	Rule loading failure	Simulate missing/invalid regexes.json.	Error is logged and page does not crash.

10. Assumptions and Constraints

Assumptions

- The extension is installed in a Chromium-based browser supporting Manifest V3.
- The targeted ChatGPT interface continues to expose a textarea or contenteditable composer.
- The configured ChatGPT hostnames remain valid for the intended deployment.
- Users understand that a warning is an indication of possible exposure rather than proof of a valid credential.
- The supplied regex patterns are maintained as project configuration and can be updated when formats change.

Constraints

- The DOM structure of ChatGPT can change, potentially requiring selector updates.
- Regular expressions cannot reliably detect every possible secret format.
- Regex-based detection can produce false positives.
- The current architecture does not verify whether a detected credential is active.
- The project has no backend for centralized rule management or telemetry.
- The extension is specifically scoped to ChatGPT and does not provide universal browser DLP.

11. Risks and Mitigation

ID	Risk	Impact/Problem	Mitigation
R1	False positives	Normal text may accidentally match a broad pattern.	Use precise patterns, test ordinary messages, and explain that matches are warnings.
R2	False negatives	A secret may use a format not represented by the rule set.	Expand the rule set over time and clearly state that detection is not exhaustive.
R3	ChatGPT UI changes	Selectors or button labels may change.	Centralize selectors and maintain regression tests.
R4	Performance degradation	Large messages and many regexes could increase scanning time.	Benchmark realistic message sizes and optimize the rule set.
R5	User bypass	A user may intentionally continue after a warning.	Make the warning clear; treat shieldGPT as a prevention aid, not an enforcement system.
R6	Sensitive test data	Test files could accidentally contain real credentials.	Use only intentionally fake or revoked examples.
R7	Extension privacy	An incorrectly designed extension could introduce privacy concerns.	Use minimal permissions and avoid external transmission of message content.

12. Security and Privacy Considerations

Security is the primary purpose of shieldGPT, but the extension itself must follow secure design principles.

- **Local inspection:** message content is inspected in the browser rather than uploaded to a shieldGPT backend.
- **No secret display:** the warning reports categories such as “AWS API Key” or “RSA private key” instead of showing the matched credential.
- **Minimal permissions:** the manifest requests storage permission and restricts content-script access to the two specified ChatGPT host patterns.
- **No real credentials in tests:** test data should use intentionally fake or revoked examples.
- **Rule integrity:** changes to regexes.json should be reviewed because weak or overly broad rules can affect usability.
- **Explicit user choice:** the user remains in control; shieldGPT warns and blocks only when the user rejects the warning.

13. Future Improvements

- Add a dedicated extension settings page for enabling/disabling rule categories.
- Add severity levels so high-confidence private keys receive stronger warnings than generic patterns.
- Show remediation guidance such as removing a credential before sending or rotating it if already exposed.
- Add configurable custom patterns for organization-specific secrets.
- Add unit tests for individual regex definitions and the detection engine.
- Improve ChatGPT DOM integration with robust selectors and interface regression tests.
- Add optional local-only audit information that records detection counts without storing message contents.
- Consider support for additional AI-chat websites while keeping permissions narrowly scoped.
- Add automated regression tests for false positives and false negatives.
- Provide an extension options interface for rule management.

14. Conclusion

shieldGPT is a feasible small software project with a clearly defined real-world purpose: reducing accidental exposure of credentials and other sensitive information when users interact with ChatGPT. The proposed architecture is intentionally simple, making it suitable for a focused development project while demonstrating requirements analysis, modular design, event-driven programming, configuration-driven detection, testing, risk management, and privacy-aware design.

The supplied implementation establishes the essential technical foundation: Manifest V3 configuration, a content script, configurable regular-expression rules, composer extraction, detection, user confirmation, and blocking of both keyboard and button submission. The roadmap builds on this foundation with systematic testing, refinement, documentation, and future extensibility.

The most important limitation is that pattern matching is not equivalent to complete secret detection. shieldGPT should therefore be presented as a preventive safety layer that catches recognizable indicators and gives users a final opportunity to stop an accidental disclosure—not as a guarantee that no secret can ever be submitted.

Appendix A. Proposed Project Structure

```
shieldGPT/  
■■■ manifest.json  
■■■ content.js  
■■■ regexes.json  
■■■ logo.png
```

This structure corresponds directly to the supplied source files and keeps the project small and easy to understand.

Appendix B. Manifest Design Summary

```
{  
  "manifest_version": 3,  
  "name": "shieldGPT",  
  "version": "1.0",  
  "permissions": ["storage"],  
  "host_permissions": [  
    "*/chatgpt.com/*",  
    "*/chat.openai.com/*"  
  ],  
  "content_scripts": [{  
    "matches": [  
      "*/chatgpt.com/*",  
      "*/chat.openai.com/*"  
    ],  
    "js": ["content.js"],  
    "run_at": "document_start"  
  }],  
  "web_accessible_resources": [{  
    "resources": ["regexes.json"],  
    "matches": [  
      "*/chatgpt.com/*",  
      "*/chat.openai.com/*"  
    ]  
  }]  
}
```

The complete supplied regexes.json remains a separate project file. This planning document describes its role and categories rather than reproducing all detection expressions.

Appendix C. Requirements-to-Implementation Traceability

Requirement	Implementation	Evidence/Role
FR-01 / FR-02	loadRegexes()	Loads and compiles regexes.json patterns.
FR-03	getComposerText()	Reads textarea/contenteditable content.
FR-04 / FR-05	findExposures()	Tests patterns and deduplicates titles.

Requirement	Implementation	Evidence/Role
FR-06	askUser()	Displays detected categories and asks for confirmation.
FR-07 / FR-08	checkBeforeSend() + event handlers	Returns allow/block decision and stops rejected events.
FR-09	keydown listener	Intercepts Enter while ignoring Shift+Enter.
FR-10	click listener	Detects Send/Submit button interactions.
FR-11	checkBeforeSend()	Allows empty/whitespace-only input.
FR-12	loadRegexes() catch block	Logs regex-file loading failures.

End of document.